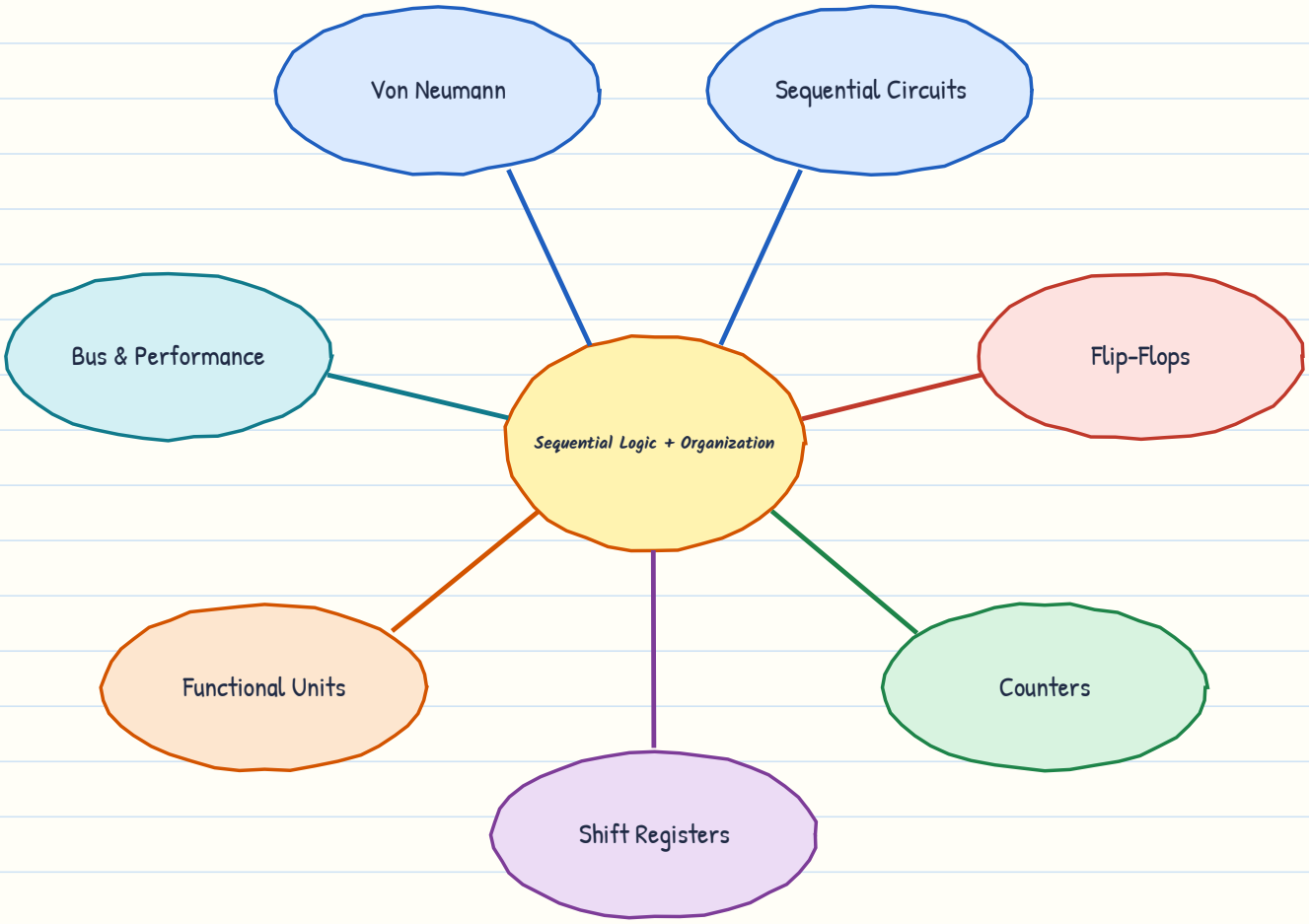
UNIT II

Sequential Circuits & Basic Structure of Computers

Latches | Flip-flops | Counters | Registers | Functional units | Buses | Von Neumann



Mind map: everything you will learn in this unit

How to use these notes

Tip: for flip-flops learn the characteristic table + the block diagram together; for computer structure just remember the block diagrams.

I. Sequential Circuits

In a ***sequential circuit*** the output depends on the ***present inputs AND the past state*** – so it needs ***memory***. Combinational circuits have no memory.

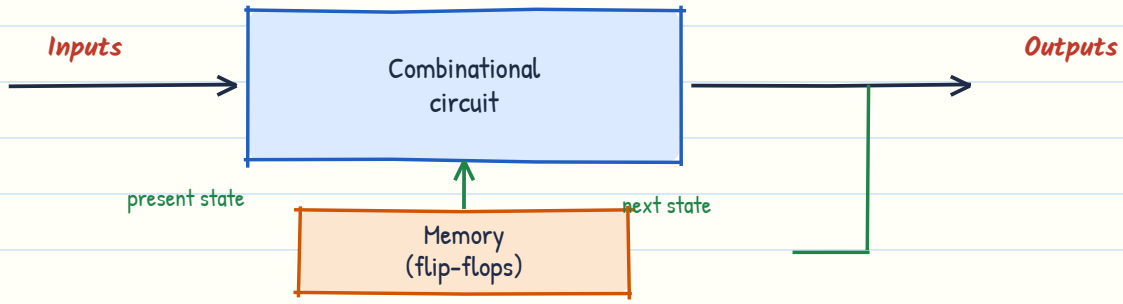


Fig 1: Sequential circuit = combinational logic + memory (feedback)

	Combinational	Sequential
Depends on	present input only	input + past state
Memory	no	yes (flip-flops)
Clock	not needed	usually clocked
Examples	adder, MUX, decoder	counter, register

- ***Synchronous:*** state changes only on a ***clock*** pulse (flip-flops).
- ***Asynchronous:*** state changes whenever inputs change (latches).

Latches – SR latch (the basic memory cell)

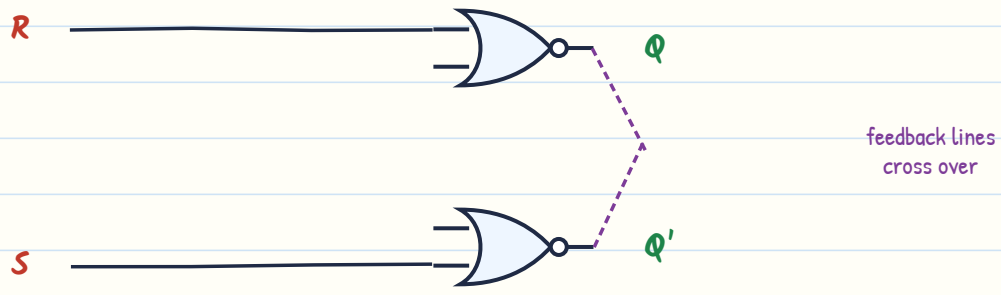


Fig 2: SR latch using two cross-coupled NOR gates

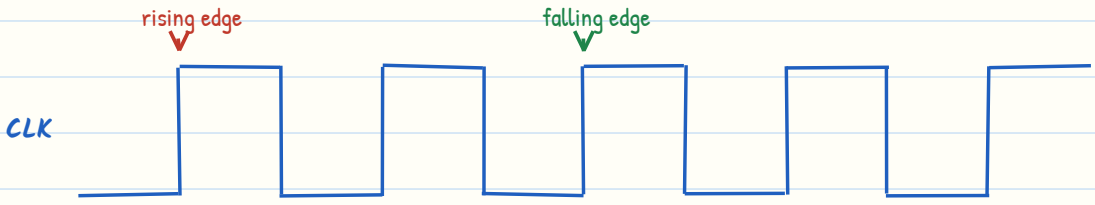
S	R	Q (next)	Action
0	0	no change	memory (hold)
0	1	0	RESET
1	0	1	SET
1	1	?	INVALID (not used)

Latch vs flip-flop

A latch is level-sensitive (changes while enable is high). A flip-flop is EDGE-triggered: it changes only at the clock edge (rising or falling) - more reliable.

2. Flip-Flops

A ***flip-flop (FF)*** is a 1-bit memory element that changes state only at a ***clock edge***. n flip-flops store n bits and give 2^n states.



FF samples input at the edge, holds value until the next edge

Fig 3: Clock signal and edge triggering

The four types of flip-flops

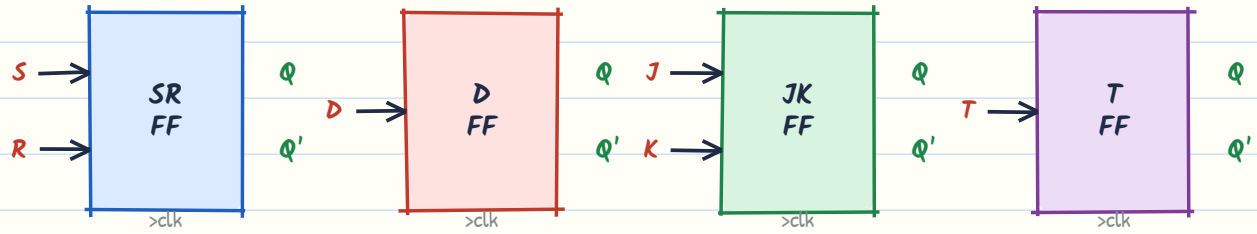


Fig 4: Symbols of SR, D, JK and T flip-flops

Type	Characteristic table	Equation
SR	00 hold, 01 reset, 10 set, 11 invalid	Q+ = S + R'Q
D	Q+ = D (copies input)	Q+ = D
JK	00 hold, 01 reset, 10 set, 11 toggle	Q+ = JQ' + K'Q
T	T=0 hold, T=1 toggle	Q+ = T xor Q

JK flip-flop (most versatile)

J	K	Q(t+1)	Meaning
0	0	Q	no change
0	1	0	reset
1	0	1	set
1	1	Q'	TOGGLE (SR's invalid state fixed)



Fig 5: JK flip-flop made from gates + master-slave idea

Concept

Race-around: in a plain level-triggered JK with J=K=1, the output toggles repeatedly while the clock is high. Cure: master-slave or edge-triggered JK.

Excitation tables (used in design)

Q	Q+	S R	D	J K	T
0	0	0 X	0	0 X	0
0	1	1 0	1	1 X	1
1	0	0 1	0	X 1	1
1	1	X 0	1	X 0	0

Excitation table answers: "to go from present state Q to next state Q+, what inputs are needed?" (X = don't care). Convert FFs: D from JK -> D = JQ'+K'Q; T from JK -> J=K=T.

3. Registers & Shift Registers

A *register* is a group of flip-flops storing a binary word (n FFs = n-bit register) - e.g. 8 D-flip-flops with a common clock make an 8-bit register.

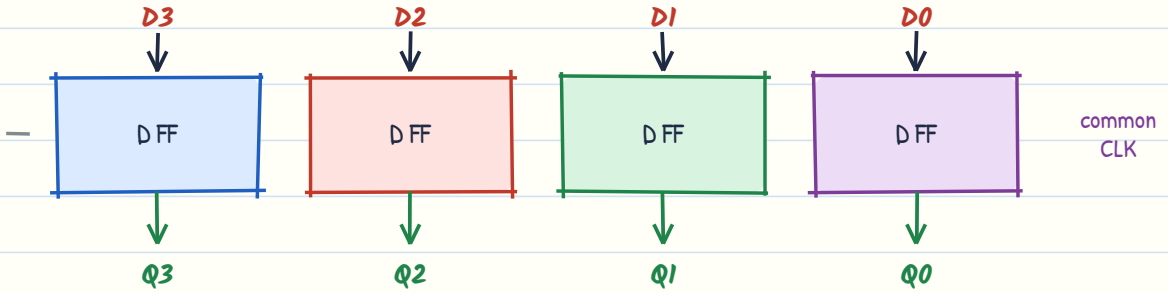
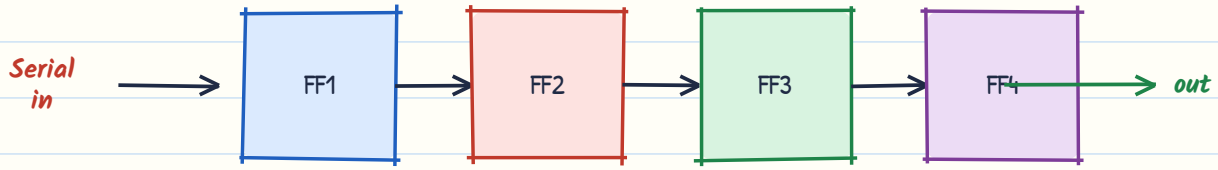


Fig 6: 4-bit parallel-load register (D flip-flops)

Shift register

A register that **shifts its bits one position** every clock pulse. Built from D flip-flops where Q of one feeds D of the next.



all FFs get the SAME clock: data moves right 1 place per pulse

Fig 7: 4-bit serial-in serial-out (SISO) shift register

Type	Input	Output	Use
SISO	serial	serial	delay line
SIPO	serial	parallel	serial -> parallel (receiver)
PISO	parallel	serial	parallel -> serial (transmitter)
PIPO	parallel	parallel	storage register
Universal	both, left/right	both	bidirectional register

Shifting example (SISO, input 1011, MSB first)

Clock	FF1	FF2	FF3	FF4
0 (reset)	0	0	0	0
1	1	0	0	0
2	0	1	0	0
3	1	0	1	0
4	1	1	0	1

- **Ring counter:** last FF output fed back to first FF - a single 1 circulates (n states).
- **Johnson (twisted-ring) counter:** inverted output fed back - 2n states.

4. Counters

A **counter** is a register that goes through a prescribed sequence of states on clock pulses. A counter with n flip-flops has up to **2^n states (modulus)**; mod-N counter counts N states.

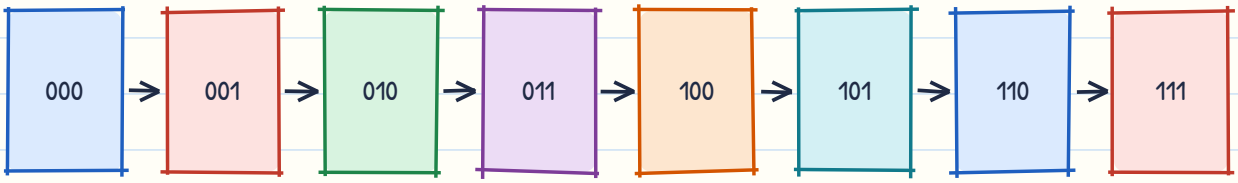
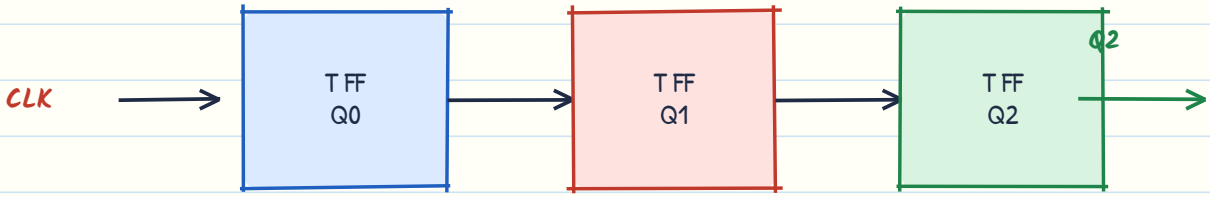


Fig 8: 3-bit binary up-counter state sequence

	Asynchronous (ripple)	Synchronous
Clock	only first FF gets clock	all FFs get same clock
Speed	slow (delay adds up)	fast
Circuit	simple	more gates
Glitches	yes	no

Ripple counter (asynchronous)



output of each FF is the clock of the next -> each stage divides frequency by 2

Q0 = CLK/2 Q1 = CLK/4 Q2 = CLK/8

Fig 9: 3-bit ripple counter using T flip-flops (T = 1)

- Count is read as Q2 Q1 Q0 (Q0 = LSB).
- **Down counter:** take clock from Q' instead of Q.
- **Mod-N (e.g. decade, mod-10) counter:** use a NAND gate to detect state N and clear (reset) all FFs.

Synchronous counter

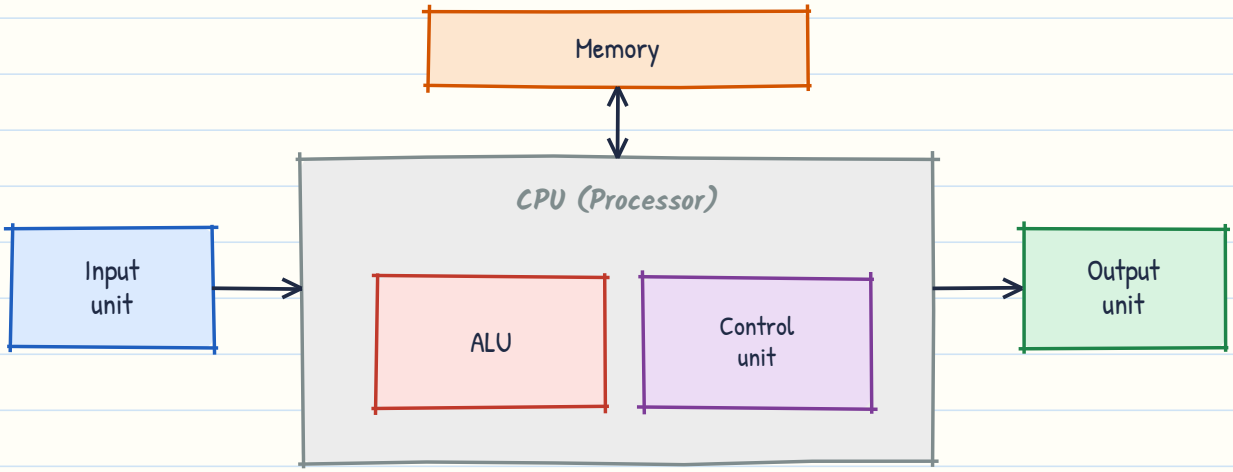
Every FF is driven by the **same clock**; toggle inputs are set by logic: T0 = 1, T1 = Q0, T2 = Q0.Q1 (a FF toggles only when all lower bits are 1). No ripple delay -> used for high speed.

5. Basic Structure of Computers

Computer types

Type	Description	Example
Embedded	built into devices, fixed task	washing machine, car ECU
Personal	single user desktop / laptop	PC, notebook
Workstation	powerful, graphics/engineering	CAD systems
Server	serves many users over network	web / database server
Supercomputer	extreme speed, parallel, science	weather forecasting
Mainframe	huge transaction processing	banks, airlines

Functional units



Secondary storage (disk, SSD): permanent, large, slow

Fig 10: Functional units of a computer

- **Input unit:** keyboard, mouse - converts data to binary.
- **Memory:** primary (RAM) for running programs; secondary for permanent storage.
- **ALU:** arithmetic and logic operations.
- **Control unit:** fetches instructions, decodes them and generates control signals for timing.
- **Output unit:** monitor, printer.

Basic operational concepts - registers and instruction cycle

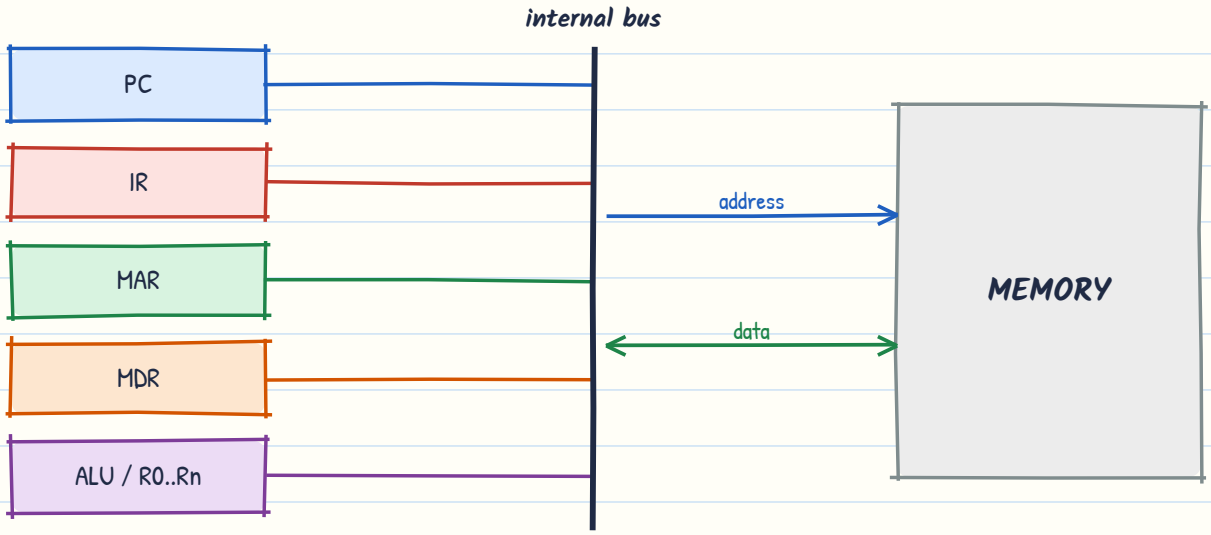


Fig 11: CPU registers connected to memory through the bus

Register	Full form	Job
PC	Program Counter	address of next instruction
IR	Instruction Register	holds current instruction
MAR	Memory Address Register	address to read/write
MDR	Memory Data Register	data being transferred
R0-Rn	General registers	temporary data
ACC / SP	Accumulator / Stack Ptr	results / stack top

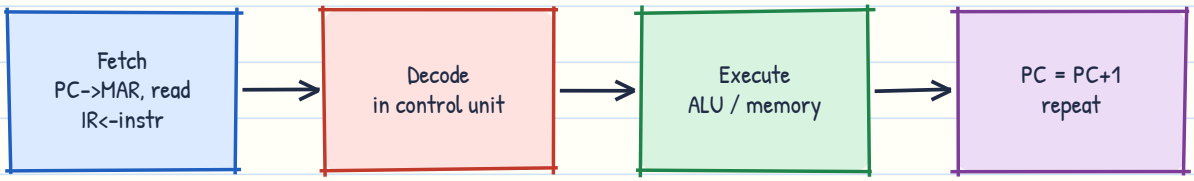


Fig 12: Instruction cycle

Bus structures

A **bus** is a set of shared wires that carries information between units. Three kinds of lines:

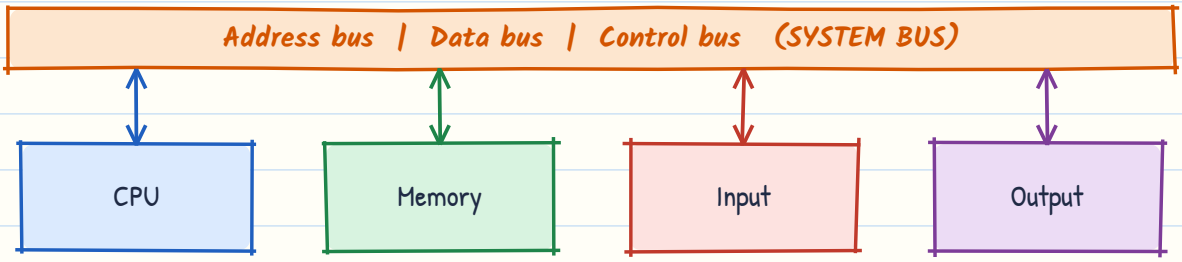


Fig 13: Single bus structure

Bus	Direction	Carries
Address	one way (CPU -> memory/IO)	location address
Data	two way	actual data / instructions
Control	both	read, write, clock, interrupt signals

- **Single bus:** cheap but only one transfer at a time (bottleneck).
- **Multiple bus:** several transfers at once, faster, costly.

Software

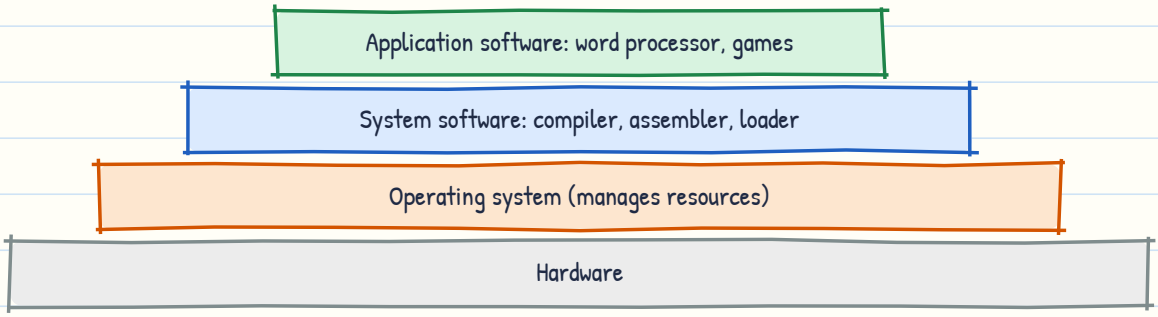


Fig 14: Layers of software

- **System software:** OS, compilers, assemblers, device drivers.
- **Application software:** user programs.
- **Compiler** translates the whole program; **interpreter** line by line; **assembler** converts assembly to machine code.

Performance

Measured by how fast a program executes. **Basic performance equation:**

$$T = (N \times S) / R$$

- T = program execution time, N = number of instructions, S = average **clock cycles per**

instruction (CPI), R = clock rate (Hz).

- To improve: reduce N (better compiler), reduce S (pipelining), increase R (faster technology).
- **MIPS** = instructions per second / 10^6; **MFLOPS** for floating point.

Multiprocessors and multicomputers

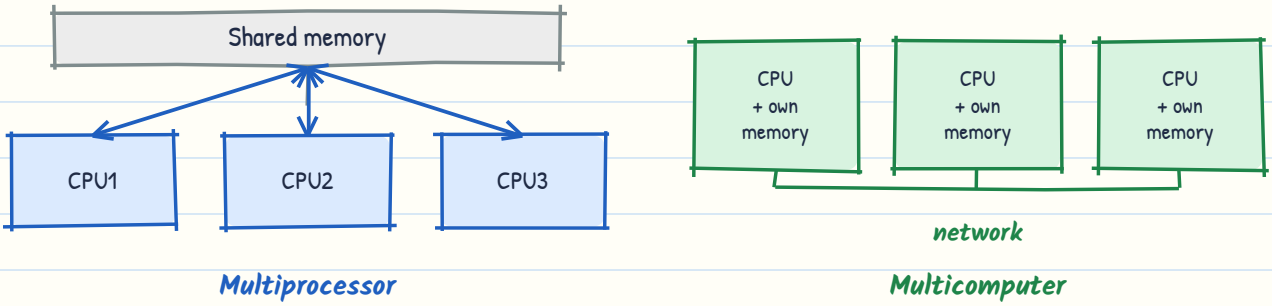


Fig 15: Multiprocessor (shared memory) vs multicomputer (message passing)

	Multiprocessor	Multicomputer
Memory	shared by all CPUs	each computer has private memory
Communication	via shared variables	message passing
Coupling	tightly coupled	loosely coupled

6. Computer Generations & Von Neumann Architecture

Gen	Period	Technology	Example
1st	1940-56	Vacuum tubes	ENIAC, UNIVAC
2nd	1956-63	Transistors	IBM 1401
3rd	1964-71	Integrated circuits (IC)	IBM 360
4th	1971-90s	Microprocessors (VLSI)	Intel 4004, PC
5th	present	ULSI, AI, parallel	AI systems

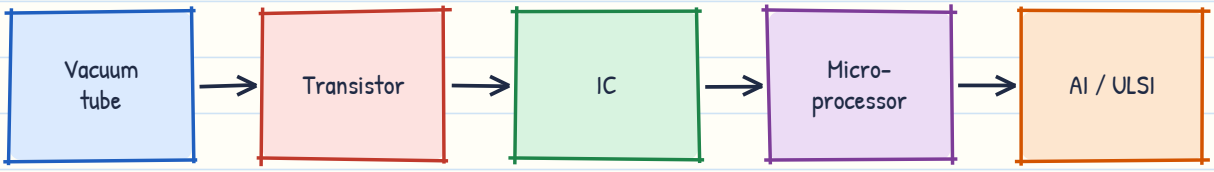


Fig 16: Evolution

Von Neumann architecture (stored-program concept)

Proposed by John von Neumann: ***instructions and data are stored together in the same memory***, and the CPU fetches them one by one.

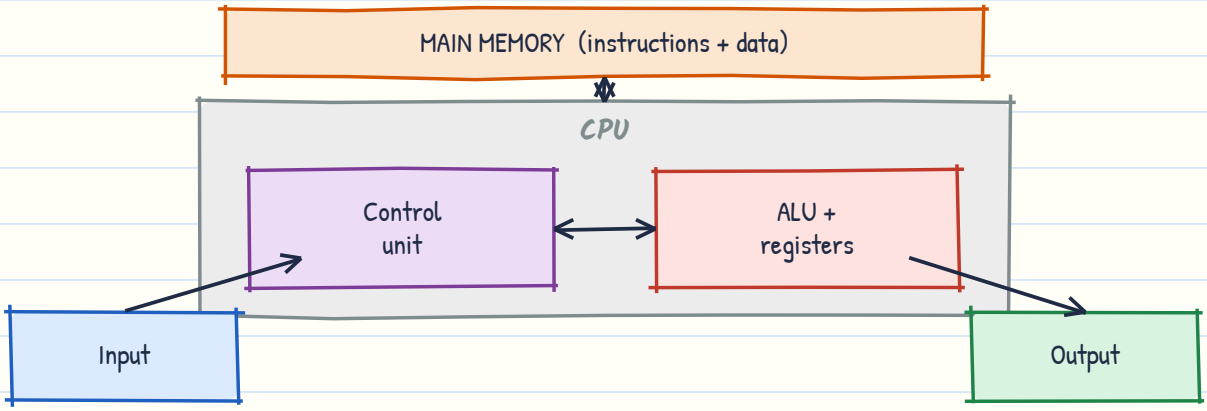


Fig 17: Von Neumann architecture

- One memory and one bus for both instructions and data -> ***Von Neumann bottleneck*** (cannot fetch both at once).
- ***Harvard architecture:*** separate memories and buses for instructions and data (faster; used in DSPs, cache split).

	Von Neumann	Harvard
Memory	single	separate
Bus	one	two
Speed	slower	faster
Cost	low	higher

Quick Revision & Exam Questions

Topic	Memory line
Sequential	output = f(input, past state); needs memory
JK 11	toggle
T FF	toggle when T=1; counters
n-bit counter	2^n states; ripple = async
PC/IR/MAR/MDR	next addr / instruction / address / data
T = N x S / R	performance equation
Von Neumann	one memory for program + data

Likely exam questions

- Differentiate combinational and sequential circuits.
- Explain SR, D, JK and T flip-flops with characteristic tables.
- Explain a shift register and its types; draw a 4-bit SISO register.
- Design / explain a 3-bit ripple counter with timing.
- Explain functional units of a computer with a block diagram.
- Explain bus structures and the basic performance equation.
- Explain Von Neumann architecture; compare with Harvard.
- Write about generations of computers.